

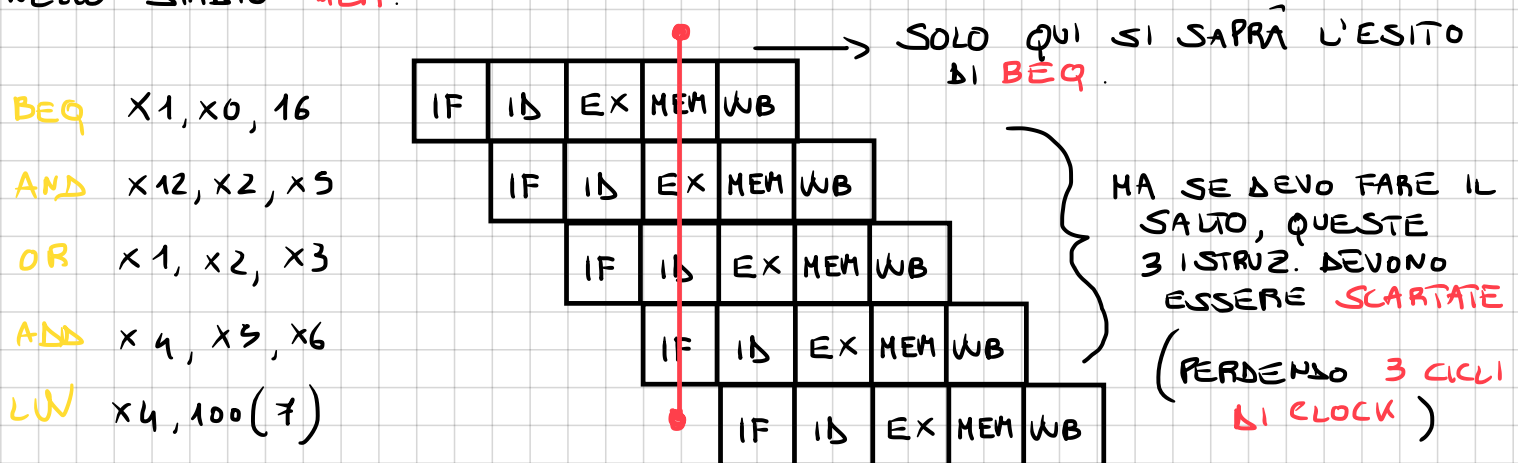
HAZARD SUL CONTROLLO:

FINO AD ORA ABBIAMO IPOTIZZATO CHE IL PROCESSORE ESEGUA SEMPRE L'ISTRUZIONE SUCCESSIVA, TUTTAVIA NON È SEMPRE COSÌ IN CASO DI IF O LOOP, OVVERO LE ISTRUZIONI DI SALTO CONDIZIONATO (COME BEQ).

L'HAZARD SUL CONTROLLO SI VERIFICA PERCHÉ IL PROCESSORE NON SA ANCORA:

- LA CONDIZIONE È VERA O FALSA.
- QUAL È L'INDIRIZZO DI DESTINAZIONE.

IN UN'ARCHITETTURA RISC-V (NON OTTIMIZZATA), L'ESITO DEL CONFRONTO E IL SEGNALE DI CONTROLLO (PCSRC) CHE DECIDE IL NUOVO PC AVVIENE NELLO STADIO MEM.



COSA SUCCEDDE SE DEVO FARE IL SALTO?

IL PROCESSORE AVVIA UNA FASE DI PULIZIA:

- VIENE AZZERATO IL CONTENUTO DEI REGISTRI DI PIPELINE (IF/ID, ID/EX, EX/MEM).
- I SEGNALE DI CONTROLLO VENGONO NEUTRALIZZATI (TRASFORMATI IN NOP).
- AL CICLO DI CLOCK 5 IL PC VIENE AGGIORNATO CON L'INDIRIZZO CORRETTO.

SOLUZIONE: SPOSTARE IL SALTO ALLO STADIO ID.

L'ARCHITETTURA VIENE MODIFICATA PER ANTICIPARE LA DECISIONE SUL SALTO ALLO STADIO ID (RECONFIG), ANTICIPANDO 2 CALCOLI:

- 1) CALCOLO DELL'INDIRIZZO DI DESTINAZIONE: IL VALORE DI PC E DELL'IMMEDIATO SONO GIÀ PRESENTI NEL REGISTRO IF/ID, QUINDI È SUFFICIENTE

SPOSTARE IL SOMMATORE CHE CALCOLA L'INDIRIZZO, DALLO STADIO EX ALLO STADIO ID, IL CALCOLO VIENE FATTO PER OGNI ISTRUZIONE (ANCHE QUELLO NON AL SALTO).

- 2) VALUTAZIONE DELLA CONDIZIONE LOGICA: PER VERIFICARE L'UGUAGLIANZA TRA 2 REGISTRI SI PUÒ USARE UNA TECNICA MOLTO VELOCE E LEGGERA:
- 1) XOR BIT-A-BIT TRA I 2 OPERANDI LETTI DAL REGISTER FILE.
 - 2) OR A CASCATI SU TUTTE LE USCITE DELLO XOR
 - 3) SE L'USCITA FINALE È ZERO, ALLORA I 2 REGISTRI SONO UGUALI.

IN QUESTO MOMENTO, SE IL SALTO VIENE PRESO, DOBBIAMO SCARTARE 1 SOLA ISTRUZIONE.

TUTTO CIÒ RISOLVE L'HAZARD SUL CONTROLLO PERÒ CREA NUOVI HAZARD SUI DATI. INFATTI LE ISTRUZIONI PRECEDENTI A BEQ POTREBBERO NON ANCORA FINITO DI SCRIVERE I REGISTRI CHE USA BEQ NEL CONFRONTO.

CI SONO 3 CASI:

- 1) ALIQUOTI IMMEDIATI. PRECEDENTE AL BRANCH:

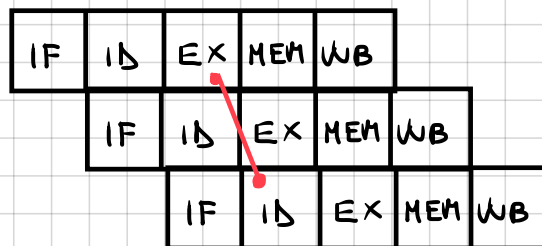
SUB x1, x2, x3
BEQ x4, x1, LOOP



x1 È PRONTO ALLA FINE DI EX HA SERVE ALLO STADIO ID

SOLUZIONE:

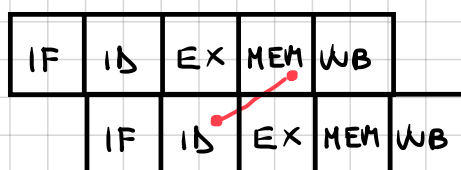
SUB x1, x2, x3
NOP
BEQ x4, x1, LOOP



DOBBIAMO PER FORZA METTERE 1 STALLO

- 2) ISTRUZ. LOAD IMMEDIAT. PRECEDENTE AL BRANCH:

LOAD x1, 1(x2)
BEQ x4, x1, LOOP



x1 È PRONTO ALLA FINE DI MEM HA SERVE ALL' INIZIO DI ID

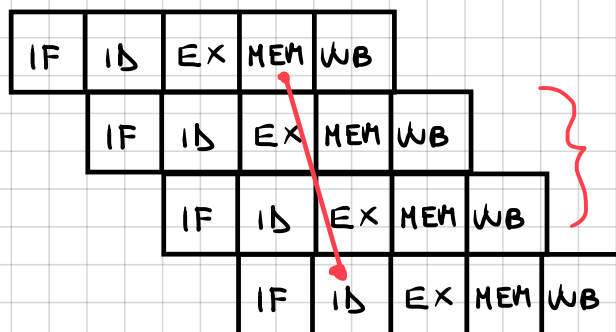
SOLUZIONE:

LOAD x1, 1(x2)

NOP

NOP

BEQ x4, x1, LOOP



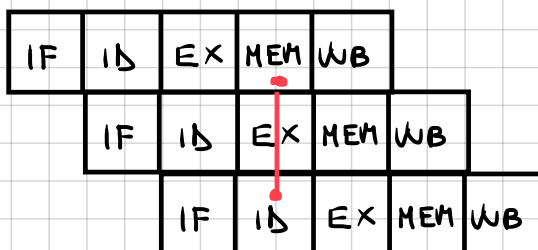
BISOGNA PER FORZA INSERIRE 2 STALLI.

3) ISTRUZ. LOAD 2 POSIZIONI PRIMA DI BRANCH:

LOAD x1, 1(x2)

ADD x2, x2, x3

BEQ x4, x1, LOOP



SEMPRE LO STESSO PROBLEMA DEL CASO 2

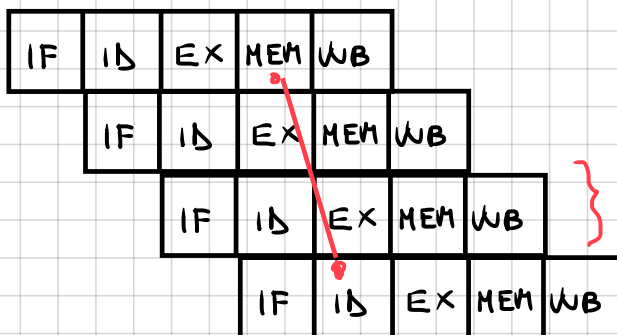
SOLUZIONE:

LOAD x1, 1(x2)

ADD x2, x2, x3

NOP

BEQ x4, x1, LOOP



DOBBIAMO INSERIRE ALMENO 1 STALLO

BISOGNA QUINDI AGGIORNARE LE CONDIZIONI SULL'UNITÀ DI RILEVAMENTO

HAZARD:

SE L'ISTRUZIONE ATTUALE È UN SALTO

IF (IF/ID.BRANCH) {

// CASO 1

IF (ID/EX.MEMWRITE AND ID/EX.RD != 0 AND
(ID/EX.RD == IF/ID.RS1 OR ID/EX.RD == IF/ID.RS2)) 1-STALLO();

// CASO 3

IF (EX/MEM.MEMREAD AND EX/MEM.RD != 0 AND
(EX/MEM.RD == IF/ID.RS1 OR EX/MEM.RD == IF/ID.RS2)) 1-STALLO();

// CASO 2

IF (ID/EX.MEMREAD AND ID/EX.RD != 0 AND
(ID/EX.RD == IF/ID.RS1 OR ID/EX.RD == IF/ID.RS2)) 2-STALLI();

}

DOBBIAMO METTERE UN MULTIPLEXER PRIMA DEL BLOCCO XOR/OR NELLO STADIO
ID GUIDATI DA PROPAGAA E PROPAGAB

	CODICE	SORGENTE
PROPAGAA	00	IF/ID
PROPAGAA	10	EX/HEM $\rightarrow$ 1 CICLO DI CLOCK
PROPAGAA	01	MEM/WB $\rightarrow$ 2 CICLI DI CLOCK
PROPAGAB	00	IF/ID
PROPAGAB	10	EX/HEM
PROPAGAB	01	MEM/WB

NEW UNITÀ DI PROPAGAZIONE:

1) SE L'ISTRUZIONE IN EX/HEM SCRIVE SU UN REGISTRO UTILE ALLA BEQ
(CASO ALU PRIMA DI BEQ - CASO 1)

IF (EX/HEM.REGWRITE AND EX/HEM.RD != 0 AND (EX/HEM.RD == IF/ID.RS1))
PROPAGAA = 10;

IF (EX/HEM.REGWRITE AND EX/HEM.RD != 0 AND (EX/HEM.RD == IF/ID.RS2))
PROPAGAB = 10;

2) SE L'ISTRUZIONE IN MEM/WB SCRIVE SUL REGISTRO UTILE A BEQ
(CASO 2 E CASO 3) OVVIAMENTE COMANDO CHE L'ISTRUZIONE IN EX/HEM
SIA + RECENTE.

IF (MEM/WB.REGWRITE AND MEM/WB.RD != 0 AND MEM/WB.RD == IF/ID.RS1
AND NOT (EX/HEM.REGWRITE AND (EX/HEM.RD == IF/ID.RS1)))
PROPAGAA = 01;

IF (MEM/WB.REGWRITE AND MEM/WB.RD != 0 AND MEM/WB.RD == IF/ID.RS2
AND NOT (EX/HEM.REGWRITE AND (EX/HEM.RD == IF/ID.RS2)))
PROPAGAB = 01;

PER ELIMINARE L'ISTRUZIONE IN IF SI INTRODUCE IF.FLUSH CHE
INTRODUCE 1 STALLO DI 1 CLOCK, AZZERA IF/ID TRASF. L'ISTRUZ. IN NOP.

TECNICHE DI PREDIZIONE DEI SALTI:

PER ORA ABBIAMO RIBOTTO IL COSTO DEL SALTO CONDIZIONATO, PER EVITARE SALTI CONTINUI, SI USANO SOFISTICATE TECNICHE DI PREDIZIONE DINAMICA DEI SALTI:

- 1) **PREDIZIONE AD 1 BIT:** SI USA UNA TABELLA IN MEMORIA CHE CONTIENE 1 SINGOLO BIT: 1 (SE IL SALTO È STATO PRESO L'ULTIMA VOLTA) 0 (SALTO NON PRESO L'ULTIMA VOLTA).

ITERAZIONE	STATO PRIMA	PREDIZIONE	SALTO REALE	ERRORE	STATO DOPO
1	0	NON PRESO	PRESO	X	1
2	1	PRESO	PRESO	✓	1
3	1	PRESO	NON PRESO	X	0

SBAGLIA COME HMMO 2 VOLTE, ALL' ENTRATA E ALLA FINE DEL CICLO.

- 2) **PREDIZIONE A 2 BIT:**

NASCE PER RISOLVERE IL PROBLEMA DEL DOPIO ERRORE ED È IMPLEMENTATO COME UNA MACCHINA A STATI FINITI A 4 STATI:

- 11 → FORTEMENTE PRESO
- 10 → DEBOLMENTE PRESO
- 01 → DEBOLMENTE NON PRESO
- 00 → FORTEMENTE NON PRESO

IN QUESTO MODO QUANDO USCIRÀ DAL PRIMO CICLO LO STATO SARÀ 11 → 10 PERMETTENDO AL PROSSIMO CICLO DI FARE MAGARI 1 SOLO ERRORE

- 3) **PREDITTORI CORRELATI:**

I PREDITTORI A 1 E 2 BIT GUARDANO LA STORIA DI QUEL SINGOLO SALTO, NEL CODICE REALE, PERÒ, SPESSO UN IF DIPENDE DALL'ESITO DI UN ALTRO IF PRECEDENTE, I PREDITTORI CORRELATI SFRUTTANO 2 STRUTTURE HARDWARE COLLEGATE:

- **BRANCH HISTORY REGISTER (BHR):** UN REGISTRO A SCORRIMENTO A N BIT CHE MEMORIZZA LA CRONOLOGIA DEGLI ULTIMI SALTI IN ASSOLUTO NEL PROGRAMMA (1101 → PRESO - PRESO - NON PRESO - PRESO).
- **PATTERN HISTORY TABLE (PHT):** UNA TABELLA AI PREDITTORI A 2 BIT, APPLICATO AL PATTERN GENERATO DAGLI ULTIMI BRANCH ESEGUITI.